
boolean



JavaScript

PRONTI, PARTENZA... VIA!

JavaScript

Tanti nomi, tante facce

1995

Netscape (chi sa cos'è?)
inventa LiveScript per dotare il proprio browser di un linguaggio di scripting che permettesse ai web designer di interagire con i diversi oggetti della pagina (immagini, form, link, ecc.)

In quello stesso anno Netscape era particolarmente vicina alla Sun Microsystems (ideatrice di Java), con cui aveva stretto una partnership.

Brendan Eich venne incaricato del progetto e rinominò LiveScript in JavaScript.

JavaScript

Tanti nomi, tante facce

1998

Esistono diverse implementazioni di Javascript quindi si decide di affidare all'ECMA (European Computer Manufacturers Association) il compito di standardizzarlo, definendo ECMAScript.
(ECMAScript è figlio di Javascript ma vengono utilizzati come sinonimi)



Una versione che ha segnato un punto di svolta è stata **ES6 (2015)**, che ha introdotto funzionalità considerate fondamentali (ad esempio `let` e `const` che noi useremo sin da subito.)

Da ES6 in poi ogni anno escono versioni nuove del linguaggio (ES2024, ES2025, ecc), che introducono man mano piccole funzionalità e miglioramenti.

JavaScript

Frontend o Backend?

È stato pensato per un utilizzo solamente **client-side** come “compagno di HTML e CSS” ed è stato così fino a pochi anni fa.

Nel **2010**, grazie a node.js e altri progetti, Javascript è stato “trasformato” in un linguaggio **server-side**, che quindi può essere utilizzato senza un browser e senza HTML e CSS, avendo funzionalità simili a quelli di linguaggi come PHP, JAVA, etc..



Con **client-side** si intende che il codice viene eseguito sulla macchina del client, il visitatore.



Pronti?

Partiamo con Javascript per il frontend.

JavaScript

Come abbiamo detto, JavaScript nasce per dare dinamicità alle pagine web.

Consente quindi di:

- Accedere e modificare elementi della pagina HTML
- Reagire ad eventi generati dall'utente
- Validare i dati inseriti dall'utente
- Interagire con il browser:
  - determinare il browser utilizzato
  - la dimensione del viewport
  - lavorare con i cookie



JavaScript

Di cosa abbiamo bisogno per iniziare a programmare in JavaScript?

1. un editor di testo
2. un browser

 **Vi ricorda qualcosa?**

JavaScript

Di cosa abbiamo bisogno per iniziare a programmare in JavaScript?

Adesso dobbiamo scegliere se inserire il codice JavaScript «in-line»

oppure in un file JavaScript esterno



Ho un senso di Déjà vu...

JavaScript

Hello JS

```html
<html>
<head>
</head>
<body>

  <script type="text/javascript">
    //qui va il codice Javascript
  </script>

</body>
</html>

```

index.html

JavaScript

Hello JS

```html
<html>
<head>
</head>
<body>

  <script type="text/javascript" src="myscript.js">
  </script>

</body>
</html>


```

index.html

```
//qui va il codice Javascript


// ..e comunque questo è un commento


/*
 ...e anche questo
*/
```

myscript.js

JavaScript

Tre modi per dire Hello World

```html
<html>
<head>
</head>
<body>

 <script type="text/javascript">
   document.writeln('Hello World!');
 </script>

</body>
</html>

```

index.html

```html
<html>
<head>
</head>
<body>

 <script type="text/javascript">
   alert('Hello World!');
 </script>

</body>
</html>


```

index.html

```html
<html>
<head>
</head>
<body>

  <script type="text/javascript">
  document.getElementById('mio_id').innerHTML=
  'Hello World!';
  </script>

</body>
</html>

```

index.html

JavaScript

È sensibile

JavaScript è **«case sensitive»,** cioè fa distinzione tra lettere maiuscole e minuscole.

Molte keyword sono costituite da parole composte unite senza alcuno spazio o trattino ad esempio: **getElementById()**

Questo metodo di scrittura è detto **camelCase**, le lettere maiuscole all'interno della keyword servono per individuare facilmente l'inizio di una parola.



```
let supercalifragilistichespiralidoso = 10;
let salagadulamegicabula = supercalifragilistichespiralidoso  * 1500 - 30;
```



LIVE CODING



JS - Variabili

Le fondamenta della programmazione

Variabili

Le fondamenta della programmazione

Come possiamo conservare dei dati?



1. Una Variabile è un contenitore
2. nel quale salviamo una informazione

Fino a quando il programma rimane in esecuzione



A differenza di un database,

i dati delle variabili non vengono salvati permanentemente.

Variabili

Le fondamenta della programmazione

Che cosa possiamo fare con una variabile?

Gli utilizzi sono infiniti, facciamo qualche esempio:

- Conservare l'input dell'utente in un form
- Salvare l’output di comandi di sistema (ad. es il giorno corrente)
- Contare eventi
- ...

Variabili

Le fondamenta della programmazione

Come faccio a dire a JS che voglio creare una variabile?

Le variabili in **JavaScript** sono definite attraverso una specifica **parola chiave** seguita dal **nome della variabile**.

Queste parole chiave sono tre e ognuna di queste conferisce alla variabile una particolare caratteristica:

const, let, var

```
1 //dichiarazione/creazione di una variabile
2 const miaVariabile1;
3 let miaVariabile2;
4 var miaVariabile3;
5
```

myscript.js

Variabili

Le fondamenta della programmazione

Come devono essere i nomi delle variabili?

I nomi delle variabili devono rispettare precisi criteri:

- devono cominciare sempre con una lettera, mai con un numero
- in generale non devono essere presenti caratteri diversi da lettere, numeri o underscore (_)
- i nomi devono essere diversi dalle keyword riservate di JavaScript

Notazione

E' molto comune utilizzare la notazione **camelCase** 🐫 per cui scrivere i nomi delle variabili unendo tutte le parole tra loro, ma lasciando le loro iniziali maiuscole, eccetto per la prima lettera. Ad es. ***questaFunzioneCheFaCose***

Variabili

Le fondamenta della programmazione

Ci sono 3 azioni che caratterizzano la definizione di una variabile:

- dichiarazione: creare una variabile per la prima volta chiamandola con un nome (utilizzando le parole chiave const o let)

- inizializzazione: creare una variabile con un valore iniziale

- **assegnazione:** dare un nuovo valore a una variabile esistente

```
1  // Dichiarazione con let
2  let firstName;
3
4  // Dichiarazione e inizializzazione con let
5  let lastName = "Rossi";
6
7  // Dichiarazione e inizializzazione con const
8  const piGreco = 3.14159;
9
10 // Dichiarazione e successiva assegnazione
11 let color;
12 color = 'yellow';
```

Nota: oltre a rispettare le regole viste prima, i nomi delle varaibili dovrebbero descrivere ciò che rappresentano.

Variabili

Quando si crea una variabile spesso contemporaneamente le si **assegna un valore**, per mezzo del segno di "uguale".
In questo modo stiamo conservando un dato che potremo riutilizzare nel nostro codice.

È anche possibile creare una variabile "vuota" e poi assegnarle un valore in un secondo momento con un'istruzione di assegnazione.

```
1 //dichiarazione di una variabile con assegnazione
2 const miaCostante = 4;
3
4
5 // dichiarazione di una variabile senza assegnazione
6 let miaVariabile;
7
8 // assegnazione successiva
9 miaVariabile = 4;
```

const

Variabili non riassegnabili

Una variabile inizializzata con la keyword **const** non è riassegnabile.

```
1 const name = 'Topolino';
2
3 name = 'Zio Paperone';
4 // Uncaught TypeError: Assignment to constant variable
```

Se proviamo quindi a mutarne il valore otterremo un errore!

Proprio perchè non è possibile riassegnare un diverso valore in futuro, non possiamo inizializzare una variabile **const** senza darle un valore iniziale

```
1 const name;
2 // SyntaxError: Missing initializer in const declaration
```

let

Variabili riassegnabili

Una variabile inizializzata con la keyword **let** è riassegnabile in futuro.

Quando si riassegna un valore a una variabile, non bisogna ripetere **let**

Una variabile con la keyword **let** può essere inizializzata senza valore e ricevere un valore successivamente.

```
1
2 // inizializzo una variabile e assegno un valore
3 let name = 'Pippo'; // Pippo
4
5 // ora cambio il valore
6 name = 'Paperino'; // Paperino
7
```

```
1
2 // inizializzo una variabile senza assegnazione
3 let name; // undefined
4
5 // ora assegno il valore
6 name = 'Paperino'; // Paperino
7
8 // posso anche cambiarlo ancora
9 name = 'Pluto';
```

let



Cosa ci aspettiamo succeda eseguendo questo blocco di codice?

```
1  // dichiarazione variabile
2  let miaStringa;
3
4  // assegnazione
5  miaStringa = 'ciao';
6
7  // // utilizzo/richiamo
8  console.log(miaStringa);
9
10 // assegnazione
11 miaStringa = 'arrivederci';
12
13 // utilizzo/richiamo
14 console.log(miaStringa);
```

let

Dichiarazione e poi utilizzo

Se proviamo ad utilizzare una variabile **let** o **const** prima delle loro dichiarazioni otterremo un errore!

```
1 console.log(name);
2 //Uncaught ReferenceError: variableNew is not
  defined
3
4 let name = 'pluto';
5
```

```
1 console.log(name);
2 //Uncaught ReferenceError: variableNew is not
  defined
3
4 const name = 'pluto';
5
```

Var

Old Style

Una variabile inizializzata con la keyword **var** ha un comportamento simile a **let**, ma presenta diversi svantaggi.

È ormai una modalità obsoleta ed è assolutamente sconsigliato utilizzarla.

Ad esempio le variabili dichiarate con **var** non danno errori se le usiamo prima di averle dichiarate, eppure il loro valore sarà undefined.

```
1
2 var variable = 10;
3 variable = 'pippo';
4 console.log(variable);
5 //pippo
```

```
1 console.log(name);
2 // undefined
3
4 var name = 'pluto';
5
```



A differenza della let e della const, la var è soggetta ad hoisting.

Hoisting, letteralmente innalzamento, è la capacità di JS di leggere prima tutte le variabili e le funzioni dichiarate nello script e consentirne l'uso anche prima della dichiarazione stessa.
Le variabili dichiarate con var risulteranno undefined ma potranno essere utilizzate senza errore. Le funzioni potranno essere utilizzate, finché siamo all'interno dello stesso scope della dichiarazione (concetto che approfondiremo più avanti).

“JS is Loosely Typed”

Troppa libertà può far male...

La **tipizzazione** è una delle caratteristiche che distinguono i linguaggi di programmazione.

Le variabili possono contenere diversi tipi di dati, e strutture di dati.
Alcuni esempi:
numeri interi (int), numeri con la virgola, stringhe, liste, etc...

In Javascript il programmatore non deve dichiarare “in anticipo” il tipo della variabile che utilizzerà.



JS automaticamente riconoscerà che Tipo è!

In altri linguaggi, è invece obbligatorio per il programmatore dichiarare immediatamente di che tipo è la variabile.

I Tipi di Dato

Il codice JavaScript manipola dei valori ed ogni valore appartiene ad un Tipo.

I Tipi JavaScript sono:

Number - Numero

String - Stringa, sequenza di caratteri

Boolean - true|false, 1|0

Object - Oggetto

Function - Funzione, un Oggetto «speciale»

Undefined - Non Definito

Null - Valore Nullo

Stringhe

I testi

Quando in JavaScript si racchiudono parole, caratteri o numeri tra virgolette oppure apici si creano delle stringhe. Generalmente il contenuto di una stringa viene salvato in una variabile.

Le stringhe possono essere unite con il segno **+**.
Questa operazione si chiama concatenazione ed il + prende il nome di **operatore di concatenazione.**

I numeri all'interno delle stringhe sono trattati come semplici caratteri di testo.

```
1  // esempio di stringhe
2  const stringaVirgolette = "Stringa tra virgolette.";
3  const stringaApici = 'Stringa tra apici';
4
5
6
7  // concatenazione di stringhe
8  const miaStringa = "Rosso" + "Acceso";
9  alert(miaStringa); // cosa uscirà?
10
11
12
13 // numeri o stringhe?
14 const stringa = '8'; //questa è una stringa
15 const numero = 8; //questo è un numero
```

Stringhe

Template Literal

Esiste un'altra modalità per concatenare le stringhe e interpolare valori senza troppa fatica.

1. Utilizziamo i **backtick** al posto delle virgolette.
2. Per incorporare una variabile nella nostra stringa usiamo la sintassi (interpolazione) **${**nomeVariabile**}**

Possiamo anche scrivere su più righe!

```
let string = 'Qui';
let stringTwo = 'Qua';
console.log(
  `
   ${string} Quo ${stringTwo}
   Andarono al mercato
  `
);


//Qui Quo Qua
//Andarono al mercato
```



La mia tastiera non ha il Backtick!!!

- **Tastiera Italiana WINDOWS :**

  Alt + 96

- **Tastiera Italiana MacOS:**

  ⌥ + \

- **Tastiera Italiana Linux:**

  Alt Gr + '

Numeri

Le variabili possono contenere numeri.

In JavaScript è possibile effettuare operazioni aritmetiche sia direttamente tra numeri, sia tra numeri e variabili che contengono numeri.

```
// addizione, con il segno +
let somma = 4 + 4;

// sottrazione, con il segno -
let sottr = somma - 3;

// divisione, con il segno /
let divisione = somma / 2;

// moltiplicazione, con il segno*
let molt = sottr * 2;
```

Le Espressioni

Concetto di espressione in programmazione

Le espressioni sono combinazioni di valori, variabili e operatori che JavaScript valuta e risolve in un singolo valore.

Alcuni tipi tipi di espressione in JavaScript:
- **espressioni aritmetiche:** utilizzano operatori matematici per calcolare un valore numerico, ad esempio *5 * 10* o *a - b*
- **espressioni di stringa:** combinano stringhe usando l'operatore +, come in *'Hello' + ' ' + 'World!'*
- **espressioni logiche:** utilizzano operatori logici per valutare condizioni, ad esempio *x > 5 && y < 10* (che vedremo più avanti)

Proprietà dei dati

Accedere alle proprietà

Ogni tipo di dato in JavaScript ti dà accesso a delle proprietà (attributi e metodi) utili. Per accedere a queste proprietà basta aggiungere un punto `.` davanti al tipo di dato.

Alcune di queste proprietà rappresentano delle *funzioni* (che vedremo più avanti) e come tali vanno invocate tramite parentesi tonde () alla fine.

Ad esempio, un paio di proprietà utili per i numeri sono `.toFixed()` e `.toString()`

Provatele dalla console e dite che risultato ottenete!

```
1  0.12345.toFixed(2);
2
3  0.12345.toString();
4
```

🤓 *Esistono moltissime proprietà ed è importante conoscerne le principali, per il resto controllate sempre in documentazione!!*



LIVE CODING



ESERCITAZIONE